

AI-Bounty-Hunter

Autonomous GitHub Bounty Issue Analysis and Pull Request System

VITyarthi Project Report

Student Name	Kartik goyal
Registration No.	26BAI11068
Course / Subject	Problem solving and analysis
Submission Date	30 September 2026

Submitted as part of the VITyarthi project requirements.

1. Introduction

AI-Bounty-Hunter is a Python-based autonomous software-engineering workflow. It searches GitHub for suitable open issues, stores candidate work in a JSON queue, analyzes a selected issue through five specialized AI roles, validates a generated patch, applies the patch to the exact target file, and automates Git branch, commit, push, and Pull Request creation.

2. Problem Statement

Open-source contribution workflows can require repeated manual steps: finding a suitable issue, understanding the repository, planning a solution, writing code, checking the patch, applying it safely, and preparing a Pull Request. The project addresses this workflow by combining issue discovery, AI-assisted analysis, controlled patching, and GitHub automation in one modular system.

3. Objectives

- Automate discovery of candidate GitHub bounty issues.
- Maintain a persistent queue of candidate targets.
- Separate planning, coding, inspection, QA, and PR-writing into distinct AI roles.
- Apply code changes only when patch matching is unambiguous.
- Automate Git branch, commit, push, and Pull Request creation.
- Demonstrate modular architecture, validation, testing, and version-control practices.

4. Functional Requirements

FR	Requirement
FR1	Discover open GitHub issues matching bounty-related criteria.
FR2	Store and process targets through a JSON queue.
FR3	Run the five-stage AI pipeline: Architect, Coder, Inspector, QA, PR Manager.
FR4	Apply a validated SEARCH/REPLACE patch to an exact repository file.
FR5	Create a Git branch, commit, push, and Pull Request after successful validation.

5. Non-Functional Requirements

NFR	Requirement
Performance	Network calls use configurable timeouts and bounded retry behavior for transient AI failures.
Security	Secrets are read from environment configuration; <code>.env` is excluded from Git; tokens are not embedded in clone URLs or normal logs.</code>
Reliability	Pipeline failures, timeouts, API failures, invalid patches, and rejected QA results stop the target instead of being marked successful.
Maintainability	GitHub, AI, configuration, patching, discovery, and orchestration are separated into modules.
Error handling	API, file, JSON, subprocess, timeout, and validation errors are handled with explicit failure paths.
Resource efficiency	The queue limits work items and repository/API requests use bounded limits and timeouts.

6. System Architecture

GitHub Issues	Scout	bounty_queue.json	Auto Hunter
GitHub Client	Architect	Coder	Inspector
QA	Patch Engine	Git branch + commit	Pull Request
PR Manager	GitHub	Result	

The architecture separates external services from local orchestration. GitHub is responsible for issue and repository operations, Gemini supplies AI analysis, the queue provides lightweight persistence, and the patch engine enforces safe source-code modification.

7. Process Flow / Workflow

Start	Scout	Queue	Select target
Fetch exact issue	Architect	Coder	Inspector
QA	Approved?	Apply patch	Commit
Push branch	Create PR	Completed	

The queue processor records a target as completed only when the main process returns success and emits a Pull Request URL. Timeouts and failures remain non-completed states.

8. UML / Design Diagrams

Use Case: the student/developer configures the system, starts issue discovery, and starts autonomous processing; GitHub and the Gemini API act as external systems.

Sequence: Scout searches GitHub and writes the queue; Auto Hunter selects a target; GitHub Client obtains repository/issue data; AI agents produce and review the patch; Patch Engine applies it; Git operations create the Pull Request.

Class/Component: Config supplies settings to the GitHub and AI clients; Auto Hunter orchestrates the pipeline; Patch Engine provides controlled source modification.

9. Storage Design

The project uses `bounty\_queue.json` rather than a relational database. Each record contains repository name, issue title, issue URL, exact issue number, status, and an optional reason. This is sufficient for the project's small persistent work queue.

10. Design Decisions & Rationale

- Five AI roles are separated so planning, coding, inspection, QA, and PR communication have distinct responsibilities.
- SEARCH/REPLACE patches are used instead of allowing unrestricted file rewriting.
- Exact issue numbers are stored in the queue so Auto Hunter does not accidentally process a different issue from the same repository.
- Repository default branch is detected through GitHub instead of hardcoding `main`.
- Configuration is environment-based to keep credentials outside source control.
- Unit tests focus on safety-critical patch and target-resolution behavior.

11. Implementation Details

The main orchestration is in `main.py`. `scout.py` performs issue discovery. `auto\_hunter.py` controls queue execution and timeout classification. `utils/api\_client.py` provides the five AI-agent methods. `utils/github\_client.py` handles GitHub REST operations and Git subprocess operations. `utils/patcher.py` validates and applies SEARCH/REPLACE blocks. `config/settings.py` provides configuration and simple `.env` loading.

12. Screenshots / Results

Automated validation performed for this deliverable: 10 unit tests executed, 10 passed, 0 failed. Tests cover exact patching, duplicate-match protection, multiple-file rejection, placeholder rejection, target-file safety, and Auto Hunter result classification.

A smoke test also verified that `main.py` starts and handles unavailable GitHub connectivity by returning a controlled failure instead of crashing. A live end-to-end GitHub/Gemini execution was not claimed because this validation environment did not provide external network access or API credentials.

13. Testing Approach

The project uses Python's built-in `unittest` framework. Safety-sensitive logic is tested without modifying a real repository. Temporary files are used for patch tests. The main entry point is also smoke-tested with credentials removed, verifying controlled error handling.

14. Challenges Faced

- Coordinating multiple AI stages while preserving clear pass/fail conditions.
- Preventing an AI-generated patch from silently modifying an ambiguous file.
- Handling GitHub API and AI API failures without exposing credentials.
- Ensuring queue processing records failures correctly.
- Keeping the project modular enough for testing and documentation.

15. Learnings & Key Takeaways

- AI automation needs deterministic validation around non-deterministic model output.
- Exact file and issue targeting is important for autonomous repository changes.
- Timeouts and explicit failure states are essential for reliable automation.
- Environment-based configuration is preferable to embedding credentials in code.
- Unit tests can protect the safety boundaries of an AI-assisted coding system.

16. Future Enhancements

- Add richer issue scoring and repository health metrics.
- Support validated multi-file patches with per-file safety checks.
- Add configurable human approval before pushing or creating a Pull Request.
- Add structured logs and persistent execution history.
- Add broader integration tests using mocked GitHub and Gemini APIs.
- Add CI workflow for automated testing on every GitHub push.

17. References

1. VITyarthi, Build Your Own Project – General Project Instructions & Submission Guidelines.
2. Google AI for Developers, Gemini API Models documentation (consulted for current Gemini model identifiers).
3. GitHub REST API documentation for Issues, Repositories, Git, and Pull Requests.
4. Python documentation for ``unittest``, subprocess handling, JSON, and environment configuration.